

An Automated Scan-to-Simulation Pipeline for Indoor Digital Twin Construction Using BLK360 and NVIDIA Isaac Sim

Sangmin Kim

Dept. of Electrical and Computer Engineering

Sungkyunkwan University, Suwon, Republic of Korea

also with CASELAB Co., Ltd., Anyang, Republic of Korea

kimsang.m@g.skku.edu

Byeongjun Kim

Dept. of Intelligent Robotics

Sungkyunkwan University, Suwon, Republic of Korea

also with CASELAB Co., Ltd., Anyang, Republic of Korea

byeongjunkim@g.skku.edu

Tae-Yong Kuc

Dept. of Electrical and Computer Engineering

Sungkyunkwan University, Suwon, Republic of Korea

Corresponding author: tykuc@skku.edu

Abstract—Constructing a simulation environment that faithfully reproduces a real indoor workspace is a major bottleneck when deploying digital twins for mobile robots: scenes are typically modeled by hand, which is slow, subjective, and quickly outdated. This paper presents an automated scan-to-simulation pipeline that converts a registered terrestrial laser scan of an industrial indoor space into a physics-enabled, photorealistic NVIDIA Isaac Sim environment, and couples it with a real mobile manipulator to form a live bidirectional digital twin. A Leica BLK360 point cloud of 68.5 million points is automatically processed into (i) a 2D occupancy grid via floor-plane estimation and height-band rasterization, (ii) a collidable wall geometry layer for physics simulation, and (iii) a dense colored point-cloud layer for visual fidelity. A swerve-drive mobile manipulator modeled from 2D CAD drawings is embedded in the scene, and a ROS 2 DDS bridge mirrors the real robot pose at 30 Hz while navigation goals selected inside the simulator are dispatched to the on-board navigation stack. A one-shot anchor-based calibration aligns the robot localization frame with the scan frame. The full conversion completes in under 80 seconds on a desktop workstation, and the resulting 3.7-million-point scene simulates at 5.6 times real time in physics-only mode and above 100 rendered frames per second in the interactive viewport. On-site experiments with the physical robot show that the reconstructed scene preserves metric scale to within 2% of tape-measured room dimensions, that end-to-end twin positioning agrees with tape-measured checkpoint distances to 5.5 cm mean absolute error, and that navigation goals issued inside the simulator drive the real robot to the commanded locations.

Index Terms—digital twin, terrestrial laser scanning, scan-to-simulation, mobile robot, NVIDIA Isaac Sim, ROS 2

I. INTRODUCTION

Digital twins—virtual replicas of physical assets that stay synchronized with their real counterparts—are increasingly adopted in manufacturing and logistics for commissioning, monitoring, and operator training of mobile robot fleets [1]. A prerequisite for any robot-level digital twin is a simulation environment that reproduces the geometry of the real workspace with sufficient fidelity: collision surfaces must be metrically

accurate for navigation and motion planning, and the visual appearance must be recognizable for human operators.

In practice, such environments are still largely modeled by hand. CAD floor plans, when available, rarely reflect the as-is state of a facility, and manual 3D modeling of clutter, racks, and equipment can take days per room while introducing subjective simplifications. Terrestrial laser scanners (TLS) such as the Leica BLK360 capture dense, millimeter-accurate colored point clouds of indoor spaces in minutes [2], but a raw point cloud is not a simulation environment: it has no collision representation, no gravity-aligned reference frame, and no interface to a robot simulator.

This paper addresses the gap with an automated scan-to-simulation pipeline targeting NVIDIA Isaac Sim [3], coupled with a live-twin runtime for a real swerve-drive autonomous mobile manipulator robot (AMMR). Our contributions are:

- An automated conversion from a merged E57 TLS export to a physics-ready Isaac Sim USD stage, comprising floor-plane estimation by RANSAC [4], height-band occupancy-grid rasterization, synthesis of collidable-but-invisible wall geometry, and a voxel-downsampled colored point-cloud layer for visual fidelity.
- A live bidirectional digital twin: the real robot pose is mirrored in simulation at 30 Hz over a ROS 2 [5] DDS bridge, and navigation goals picked inside the simulator are dispatched to the robot's on-board Nav2 [6] stack. A one-shot anchor-based calibration solves the SE(2) offset between the robot localization frame and the scan frame.
- A quantitative evaluation of the pipeline covering conversion time (under 80 s end to end), scene complexity, simulation throughput ($5.6\times$ real time physics-only; near real time with rendering), and geometric fidelity of the twin against tape-measured ground-truth distances.

Unlike approaches that reconstruct watertight meshes from

scans, we deliberately keep two decoupled layers—a coarse collision layer derived from the occupancy grid and a dense point layer for appearance—which keeps conversion fast, robust to scan noise, and directly navigable by a mobile base.

II. RELATED WORK

A. As-built capture and scan-to-model conversion

Automatic reconstruction of as-built environment models from point clouds has been studied extensively in the construction domain (scan-to-BIM) [7]. These methods target semantically rich building models and typically require minutes to hours per scene; their output (IFC/BIM) is not directly consumable by robot simulators. Game-engine-based reconstruction pipelines produce visually convincing meshes but seldom guarantee the metric consistency required for navigation. Our pipeline trades semantic richness for speed and robustness, producing a physics-consistent simulation stage in tens of seconds.

Table I contrasts the main routes from a captured indoor space to a robot-usable simulation environment. A direct head-to-head benchmark is not yet practical—published systems target different simulators and input modalities and rarely release end-to-end code—so the table compares the characteristics reported by each line of work against those measured for our pipeline in Section IV: our conversion is fully automatic, completes in 78.5 s, and is validated metrically both at the scene level and end to end.

TABLE I
ROUTES FROM A CAPTURED INDOOR SPACE TO A SIMULATION
ENVIRONMENT (CHARACTERISTICS AS REPORTED BY EACH LINE OF
WORK)

Approach	Autom.	Turnaround	Metric valid.
Manual modeling [1]	manual	days/room	rarely rep.
Scan-to-BIM [7]	semi-autom.	min.–hours	element-level
Mesh real-to-sim [8]	offline pipeline	not reported	visual focus
This work	automatic	78.5 s	$\leq 2\%$; 5.5 cm

B. Robot simulators and digital twins

Gazebo [9] has long been the default simulator in the ROS ecosystem; NVIDIA Isaac Sim adds GPU-accelerated PhysX simulation, OpenUSD scene composition, and photorealistic rendering, and has been adopted for robot learning [10] and sim-to-real research [11]. Existing Isaac Sim workflows, however, assume the environment is authored beforehand. On the twin side, most reported robot digital twins mirror robot state into a manually built scene. The combination presented here—an automatically scan-built Isaac Sim scene coupled bidirectionally to a real mobile manipulator—has, to the best of our knowledge, not been quantitatively characterized before.

III. SCAN-TO-SIMULATION PIPELINE

Fig. 1 shows the system, which consists of an offline conversion stage (top row) and an online twin runtime (bottom row). Offline, the merged BLK360 E57 export first passes through preprocessing (Section III-A): voxel thinning followed

by RANSAC floor-plane estimation, which fixes the gravity-aligned reference frame. From this common frame the pipeline derives three artifacts: (i) a 2D occupancy grid, obtained by rasterizing a robot-relevant height band and exported both as ROS map files and, after greedy box merging, as the invisible collision layer (Section III-B); and (ii) a dense colored point cloud, voxel-downsampled and embedded as the visual layer (Section III-C). The collision and visual layers are composed into a single Isaac Sim USD stage together with the AMMR robot model (Section III-D). Online, the real robot’s localized pose is streamed at 30 Hz through a ROS 2 DDS bridge and applied to the simulated robot, while navigation goals picked in the simulator viewport travel the reverse path to the robot’s on-board Nav2 stack; a one-shot anchor calibration aligns the two coordinate frames (Section III-E).

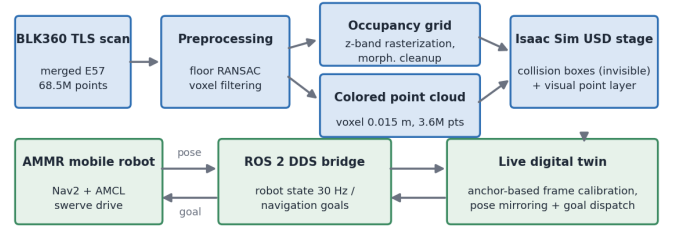


Fig. 1. System overview. Offline (top): the merged BLK360 E57 export is converted into an occupancy grid, a collidable wall layer, and a dense colored point layer composing the Isaac Sim stage. Online (bottom): a ROS 2 DDS bridge mirrors the robot pose at 30 Hz and returns navigation goals selected in the simulator.

A. Scan acquisition and preprocessing

The test space is an industrial robot test room of approximately 13×9 m. It was selected on three grounds. First, it is the actual deployment environment of the AMMR and contains genuine industrial furnishing—workbench rows, racks, equipment cabinets, and monitor clusters—so the reconstruction is exercised on the clutter distribution the twin must ultimately serve, rather than on a staged mock-up. Second, the room combines surface types that are known to stress laser-scan-based reconstruction (glass windows, dark light-absorbing equipment, and reflective flooring), making it a conservative test case for the conversion. Third, the facility allows controlled, repeatable access, which is required for the tape-measured ground-truth campaign and the on-robot experiments in Section IV; conveniently, the robot itself was present during scanning, which later provides the calibration anchor of Section III-E. The room was captured with a Leica BLK360 from multiple viewpoints; scan placement followed a visibility-coverage plan, and the registered stations were exported as a single E57 file (68.5 M points, 1.3 GB). The pipeline first estimates the dominant floor plane with RANSAC on a voxel-thinned subset (implemented with Open3D [12]), yielding the gravity-aligned reference frame and the floor height z_0 ; all subsequent layers are expressed relative to this frame, so no manual leveling or origin selection is required.

B. Occupancy grid and collision layer

Points within a height band $[z_0+0.10, z_0+1.80]$ m—the band relevant for a mobile base—are rasterized into a 2D occupancy grid at 0.05 m resolution (654×474 cells for the full scan extent). Morphological cleanup removes speckle from reflections and small blobs below 0.02 m^2 . The occupied cells are then greedily merged into axis-aligned boxes and emitted as USD cubes of 2.5 m height with PhysX collision enabled. Crucially, the collision boxes are marked *invisible*: they provide physics without occluding the visual point layer. The same occupancy grid is exported in ROS map format (PGM/YAML) so that the simulated and real navigation stacks can share one map.

C. Visual point-cloud layer

For appearance, the full-resolution colored cloud is voxel-downsampled to 0.015 m (3.6 M points, ceiling included) and embedded as USD `Points` prims with per-point color and a rendered point width of 0.013 m, chosen at approximately 0.85 times the voxel size so points visually fuse into surfaces at navigation-relevant viewing distances. The two-layer design keeps the stage lightweight: collision queries run against 1,430 boxes rather than millions of points, while the operator sees the photographic appearance of the real room.

D. Robot model and mobile-base control

The AMMR is a swerve-drive mobile manipulator with two independently steered wheel modules mounted diagonally at $p_{1,2} = (\pm 0.364, \mp 0.137)$ m and four passive casters. Its URDF (12 links, 11 joints) was authored from the manufacturer’s 2D drawings. For simulation-only operation the base is velocity-controlled through the standard inverse kinematics: for a commanded body twist (v_x, v_y, ω) each module i tracks

$$\mathbf{v}_i = \begin{bmatrix} v_x - \omega p_{i,y} \\ v_y + \omega p_{i,x} \end{bmatrix}, \quad \delta_i = \text{atan2}(v_{i,y}, v_{i,x}), \quad \dot{\varphi}_i = \frac{\|\mathbf{v}_i\|}{r} \quad (1)$$

with wheel radius $r = 0.075$ m. Steering angles are wrapped to $[-90^\circ, 90^\circ]$ with wheel-speed reversal; a hysteresis of 0.12 rad on the reversal decision and a cos speed gate suppress the flip-flopping that otherwise occurs when a module operates exactly at the wrap boundary (e.g., during pure rotation).

E. Live twin bridge and frame calibration

During live operation the simulated robot is switched to kinematic mode and mirrors the real pose. The robot publishes its localized pose (TF *map* $\rightarrow$ *base*, AMCL [13]) as a 30 Hz odometry stream; the simulator subscribes over a DDS bridge (Cyclone DDS [14], dedicated domain, unicast peer configuration over Wi-Fi) and applies the pose each physics step. In the reverse direction, dragging a goal marker in the simulator publishes a goal pose that an on-robot relay forwards to Nav2, closing the loop: the physical robot navigates to targets picked in the twin.

The robot’s SLAM map frame and the scan frame differ by an unknown planar rigid transform. We calibrate it with a single anchor: the robot is parked at a physically identifiable

spot whose scan-frame pose $(\mathbf{a}, \theta_a)$ is known—in our case the robot itself was captured in the scan, so its scanned footprint provides the anchor. On receipt of the first live pose $(\mathbf{r}, \theta_r)$ the offset is closed-form:

$$\theta_{\text{off}} = \theta_a - \theta_r, \quad \mathbf{t}_{\text{off}} = \mathbf{a} - R(\theta_{\text{off}}) \mathbf{r} \quad (2)$$

and is reused for all subsequent poses and goals. The accuracy and failure modes of this single-anchor scheme are analyzed in Section IV-F.

IV. EXPERIMENTAL RESULTS

All experiments ran on a desktop workstation (AMD Ryzen 9 9900X, 24 threads, 32 GB RAM, NVIDIA RTX PRO 4000 Blackwell 24 GB, Isaac Sim 5.1, ROS 2 Jazzy).

A. Conversion time

Table II reports single-run wall-clock times for each pipeline stage on the 68.5 M-point scan. The complete conversion from raw E57 to a simulation-ready USD stage takes 78.5 s; the dominant cost is writing the 3.7 M-point visual layer. A reduced-density variant (485 k points) completes in 36.5 s. Authoring a comparable environment by hand is a multi-day effort, and unlike manual modeling the pipeline can be rerun after every facility change.

Resource utilization is modest. The conversion runs entirely on the CPU: the floor-estimation and rasterization stage peaks at 8.1 GB of resident memory—dominated by holding the full 68.5 M-point cloud in RAM—at an average CPU utilization of 2.4 cores (measured with `/usr/bin/time`); no GPU is required until simulation. The simulator process itself occupies 3.1 GB of GPU memory with the full scene and robot articulation loaded in headless physics-only mode, leaving ample headroom on the 24 GB device for rendering and additional robots.

TABLE II
SCAN-TO-SIMULATION CONVERSION TIME (68.5 M-POINT E57)

Stage	Time [s]
Floor RANSAC, occupancy grid, ROS map export	14.8
Dense visual cloud extraction (voxel 0.015 m)	16.8
Visual layer USD export (3.7 M points)	46.8
Collision-layer synthesis (1,430 boxes)	0.1
Total	78.5

B. Scene complexity and simulation performance

The generated stage contains 1,430 collision boxes (≈ 17 k triangles) and 3,725,047 rendered points (166 MB USD). With the AMMR articulation loaded (physics at 120 Hz), headless physics-only simulation advances at 675 steps/s, a real-time factor (RTF) of 5.6; with rendering enabled the simulator sustains 51 rendered frames per second (RTF 0.84). Rates were measured as the simulation-clock publication rate over 30 s windows. In the interactive GUI session used for the on-site experiments below, the viewport sustained 104 frames per second (9.6 ms frame time) with the full point layer in view.

C. Simulation-only base control validation

Before coupling to the real robot, the swerve base was validated in simulation: straight-line, lateral (strafe), and in-place rotation commands produce odometry consistent with the commanded twist, and the flip-hysteresis of (1) eliminates the wheel-speed oscillation otherwise observed at steering-wrap boundaries. Spot checks confirmed straight-line (1.13 m forward run), lateral (1.45 m strafe), and in-place rotation (65°) maneuvers, each with odometry matching the commanded motion direction.

D. Live twin verification

The bridge contract was verified end to end with the physical AMMR in the scanned facility (Fig. 2): the 30 Hz state stream is received across the Wi-Fi link, the anchor calibration (2) converges from a single pose sample, and goals picked in the simulator drive the real robot through Nav2. During the on-site session the twin mirrored teleoperated motion continuously across a 3.4 m sweep of the room; the mirrored pose advanced at the AMCL update rate of the onboard localization. Across two sessions on consecutive days, all four navigation goals issued by dragging the target marker in the simulator viewport drove the robot to the corresponding physical locations (Fig. 3); Table III lists the individual trials. The twin-frame arrival error of 0.19–0.23 m (mean 0.21 m) is consistent with the Nav2 goal tolerance, at commanded speeds up to 0.52 m/s.

TABLE III
SIMULATOR-ISSUED NAVIGATION-GOAL TRIALS

Trial	Session	Error [m]	Note
1	day 1	0.19	14.4 s run 3.0 m straight, up to 0.52 m/s
2	day 1	0.21	
3	day 2	0.19	
4	day 2	0.23	
Mean (4/4 reached)		0.21	



Fig. 2. Test facility (left) and its generated twin during the live session (right). The yellow body with the white column is the AMMR proxy model; the red cone is the draggable navigation-goal marker.

E. Geometric fidelity

We evaluate metric accuracy at two levels: scene geometry alone, and the end-to-end twin including localization.

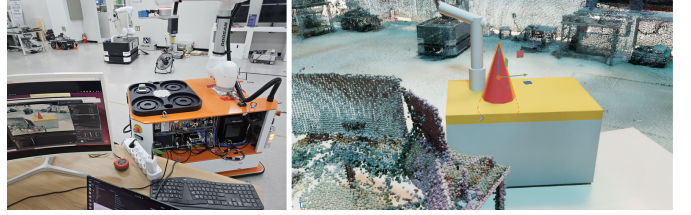


Fig. 3. Simulator-issued navigation goal. Left: the physical robot at the reached location; right: the twin after arrival, parked on the goal marker.

1) *Scene dimensions*: Three room features were tape-measured and compared against the same dimensions extracted from the twin’s point layer (histogram run-length analysis after wall-axis alignment). As Table IV shows, all errors are within 2% (mean absolute error 6.4 cm, or 1.3%), i.e., the scan-to-simulation conversion itself preserves metric scale to centimeter level.

TABLE IV
SCENE-FEATURE DIMENSIONS: TAPE MEASURE VS. TWIN POINT LAYER

Feature	Real [m]	Twin [m]	Error
Window-side wall length	8.000	7.918	−1.0%
Double-door width	1.700	1.680	−1.2%
Workbench row length	5.010	5.100	+1.8%

2) *End-to-end positioning*: We compare tape-measured distances between marked floor checkpoints against the corresponding distances reported by the twin with the robot parked on each checkpoint. This captures the full error budget—TLS registration, occupancy quantization, AMCL localization, and anchor calibration—without requiring a global frame alignment between ground truth and simulation. Over three checkpoint pairs (0.86, 3.14, and 1.30 m tape distance) the twin distances deviated by a mean absolute error of 5.5 cm (RMSE 7.4 cm, max 12.5 cm, signed bias +2.8 cm; Fig. 4). The largest deviation occurred on the segment executed with large in-place rotations and matches the AMCL repeatability we observed when revisiting the same floor marker (17 cm), indicating that end-to-end twin accuracy is dominated by the onboard localization rather than by the reconstructed scene, whose intrinsic scale error is an order of magnitude smaller (Table IV).

F. Discussion: scalability and limitations

1) *Scalability*: Every pipeline stage is linear in the number of input points, and the measured bottlenecks are I/O-bound (Table II), so conversion time scales roughly linearly with scan size; the occupancy grid already covers the full 32.7×23.7 m scan extent at 0.05 m resolution. Larger facilities enter through two paths: more TLS stations, whose registration is handled at capture time before the E57 export, and a proportionally larger visual layer, which can be bounded by raising the voxel size (the 485 k-point variant converts in 36.5 s) or by tiling the stage per room and loading tiles on demand. The main memory constraint is holding the merged cloud in RAM (8.1 GB for

68.5 M points), which for facility-scale scans beyond roughly 10^8 points would call for chunked streaming of the E57 input. For dynamically changing environments the pipeline itself offers a pragmatic mitigation—a facility change is absorbed by rescanning and rerunning the 80-second conversion—but continuous object-level scene updating from on-robot sensors remains future work.

2) *Limitations of anchor-based calibration:* The one-shot calibration of (2) inherits the localization error of the single pose sample it is computed from: any AMCL error at anchor time becomes a permanent bias of the session. In our two on-site sessions the recovered offsets differed by 0.60 m and 9.1° , reflecting run-to-run differences in AMCL initialization rather than scene error; our operating procedure therefore verifies the overlay visually after anchoring and re-anchors if the twin is misaligned. The scheme also presupposes a physically identifiable anchor pose in the scan (here, the robot’s own scanned footprint). Averaging over multiple anchor poses, or refining the offset by matching the live 2D-lidar scan against the TLS cloud, would remove the single-sample dependence.

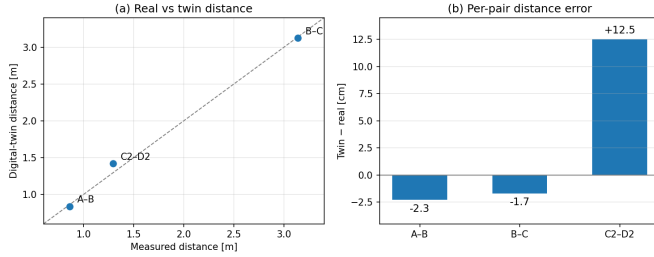


Fig. 4. Checkpoint-pair distances, tape measure vs. twin: scatter against the identity line (left) and signed per-pair error (right).

V. CONCLUSION

We presented an automated pipeline that turns a terrestrial laser scan of an indoor industrial space into a physics-enabled Isaac Sim environment in under 80 seconds, and a live bidirectional digital twin that couples the generated scene to a real swerve-drive mobile manipulator over ROS 2. The two-layer scene design (invisible collision geometry plus dense colored points) yields both metric navigability and visual fidelity while sustaining near-real-time simulation of a 3.7 M-point scene. On-site experiments confirmed that the generated scene preserves metric scale to within 2% and that end-to-end twin positioning tracks tape-measured ground truth to 5.5 cm mean absolute error, with simulator-issued goals executed by the physical robot. Future work includes automatic semantic decomposition of the scene into articulated objects, mesh reconstruction for contact-rich manipulation, continuous object-level scene updating from on-robot sensors for dynamically changing facilities, and multi-robot twins.

ACKNOWLEDGMENT

The authors thank the members of the Control and Robotics Laboratory at Sungkyunkwan University for helpful discussions and for supporting the on-site experiments.

REFERENCES

- [1] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [2] Leica Geosystems, “BLK360 imaging laser scanner,” [Online]. Available: <https://leica-geosystems.com/products/laser-scanners> (accessed Jul. 2026)
- [3] NVIDIA, “Isaac Sim,” [Online]. Available: <https://developer.nvidia.com/isaac/sim> (accessed Jul. 2026)
- [4] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [6] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A navigation system,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2020, pp. 2718–2725.
- [7] V. Pătrăucean, I. Armeni, M. Nahangi, J. Yeung, I. Brilakis, and C. Haas, “State of research in automatic as-built modelling,” *Adv. Eng. Informat.*, vol. 29, no. 2, pp. 162–171, 2015.
- [8] F. Xia, A. R. Zamir, Z.-Y. He, A. Sax, J. Malik, and S. Savarese, “Gibson Env: Real-world perception for embodied agents,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 9068–9079.
- [9] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2004, pp. 2149–2154.
- [10] M. Mittal et al., “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3740–3747, 2023.
- [11] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2020, pp. 737–744.
- [12] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv preprint arXiv:1801.09847*, 2018.
- [13] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, “Monte Carlo localization for mobile robots,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 1999, pp. 1322–1328.
- [14] Eclipse Foundation, “Eclipse Cyclone DDS,” [Online]. Available: <https://cyclonedds.io> (accessed Jul. 2026)